

Same Words, Different Procedure: Counterfactual Transfer Tests for Reusable LLM-Agent Skills

Liang Song^{1*} and Zhai Jiabao²

^{1*}Business School, Xi'an International University, Xi'an, 710077, Shaanxi, China; ORCID: [0009-0002-6109-0639](https://orcid.org/0009-0002-6109-0639).

²School of Management and Economics, North China University of Water Resources and Electric Power, Zhengzhou, 450046, Henan, China.

*Corresponding author(s). E-mail(s): liangsong.1976@126.com;
Contributing authors: zhaijiabao123@126.com;

Abstract

An agent completing a new task after receiving a reusable skill does not show that the skill caused success: the base agent may already solve the task, while semantic retrieval may select a source that repeats target nouns but prescribes the wrong state transition. We introduce the Counterfactual Skill Transfer Test (CSTT), a state-matched protocol that crosses source procedural compatibility with surface similarity on the same native target. Every source is a successful ALFWorld training trajectory compiled into a role-normalized procedure and a small executable contract. Each target is reset for no-skill, binding-only, procedure-matched near/far, and procedure-mismatched near/far conditions. The binding control shares target grounding and search safeguards but receives no source procedure, enabling separate estimates of executor-level benefit and causal skill credit. After two failed pilots and one final mechanism revision, we froze code, source assignment, three model revisions, a zero-overlap roster, inference, and decision gates. Confirmation contains 60 untouched native targets, 3 local model families, 180 paired target-model blocks, and 1080 episodes. Matched-far skills succeed on 97.8% of blocks versus 10.0% for surface-near procedural lures, an 87.8-point paired gain (95% target-stratified bootstrap CI 82.8–93.3; task-clustered randomization $p = 1.00 \times 10^{-6}$). The equal-family macro gain is 87.7 points, and the net matched-far effect over binding-only is 83.3 points with a 0.0% harm rate. CSTT is an evaluation and credit-assignment instrument, not a retrieval algorithm; evidence is limited to instance-level transfer in one embodied text environment.

Keywords: LLM agents, agent skills, skill transfer, counterfactual evaluation, procedural compatibility, negative transfer

1 Introduction

Reusable skills turn a successful agent experience into an artifact that can be retrieved for later tasks. Classical reinforcement-learning and multiagent transfer research evaluates transfer against a no-transfer target baseline and emphasizes source–target compatibility [1, 2]. Recent JAAMAS work likewise quantifies the benefit of shared experience against learning without transfer, using regret bounds in a multiagent setting [3]. Recent work abstracts implementation details, chooses skill granularity, compresses repeated experience, and adapts skill memories across tasks or domains [4–8]. These systems commonly report the downstream task success achieved with a skill. That number is operationally important, but it is not enough to credit the skill.

Two confounds matter in deployment. First, an increasingly capable base agent may solve the target without any reusable artifact. Counting that episode as transfer overstates library value. Second, a semantic retriever can prefer a source that shares objects and destinations while encoding a different procedure. A source that says “move one object” can look highly relevant to a target that requires moving two; a heat-then-place trace can repeat the target object while imposing an unnecessary state transition. Such lures can be redundant on monotone tasks, actively harmful on others, or fail only after costly interaction. Broad lifecycle studies and failure analyses show that skill utility depends on extractor, consumer, mismatch, and execution context [9, 10]. What is missing is a controlled test that separates procedure from words on the exact same native state.

We propose the Counterfactual Skill Transfer Test (CSTT). For each target, it constructs four real-source conditions crossing procedural compatibility and surface similarity. Two additional controls measure the unfiltered base agent and the same grounded executor without a source procedure. All six episodes restart the native game and must expose an identical initial observation, objective, and admissible-action signature. Transfer is then a paired outcome, not a label attached to a successful skill-conditioned episode.

The protocol deliberately uses both a far matched source and a near mismatched source. The primary comparison asks whether procedure still wins when lexical similarity favors the lure. The source artifact omits source entities and target answers: it exposes only a role-normalized operator sequence, generic procedural text, and a contract compiled from a successful source witness. At runtime, target entities come from the public objective. This construction evaluates reuse under a fixed selector; it does not claim to solve retrieval.

The work follows a strict development–confirmation separation. Two agentic pilots failed to deliver stable causal benefit. A third and final permitted revision replaced unconstrained prompt following with an auditable finite-state stage filter over native actions. A 30-target development screen passed, after which all method files, source pairs, model revisions, statistics, thresholds, and a previously untouched 60-target roster were frozen. Confirmation was run sequentially on three open local model families using one RTX 3090.

This paper contributes:

- a six-arm same-state protocol that separates skill-conditioned success, executor assistance, causal skill benefit, and harmful transfer;
- a crossed test of procedural compatibility and transparent lexical similarity using only successful real source trajectories;
- an auditable role-normalized executor that records every candidate, rejected action, stage transition, and native state signature;
- a preregistered-style confirmation with untouched targets, three model families, task-aware inference, family-balanced robustness, and retained failed development mechanisms.

We ask: **RQ1**, does a procedurally compatible but lexically far skill outperform a lexically near procedural lure on the same target? **RQ2**, does the compatible skill add benefit beyond target binding and generic search safeguards? **RQ3**, are the effects robust across model and task families, and what failure modes delimit the claim?

2 Related Work

2.1 Transferable skills

PolySkill separates an abstract goal from concrete implementations to promote generalization [4]. Break It Down, Pass It On studies task and subtask granularity and alternative text/code representations [5]. Skill Reuse as Compression uses a minimum-description length perspective to discover recurring reusable patterns [6], while SkillLens retrieves and adapts skills at multiple granularities [7]. Beyond Domains transfers web interaction patterns across sites [8]. These works improve learning, retrieval, adaptation, or downstream success. CSTT instead asks how to attribute success once a source has been supplied.

2.2 Skill generation and consumption

SKILL-DISCO compiles traces into reusable procedures [11]; SkillEvolBench tests the progression from episodic experience to procedural knowledge [12]. A systematic extractor-consumer study reports that generated skills can help or hurt depending on the model and domain [9]. CSTT complements these pipelines with a consumer-side causal endpoint. It requires successful source witnesses, but does not evaluate the quality of an induction algorithm.

2.3 Reliability and provenance

Agent Skills Can Be Harmful separates functional and efficiency regressions under skill use [10]. Multi-trace SkillTrace audits whether expressions, implementations, and operations were reused [13]. Provenance answers where an artifact came from; CSTT asks whether using that artifact changed the target outcome. The two forms of evidence are complementary: a causal benefit without provenance is hard to reproduce, while provenance without a same-target control does not establish utility.

2.4 Counterfactual skill evaluation

Counterfactual Trace Auditing aligns with-skill and without-skill traces to identify behavioral influence patterns that pass rates can hide [14]. SkillMaster uses counterfactual utility on probe tasks as a training signal for skill editing [15], while AFTER evaluates procedural-memory transfer across tasks, roles, and model backbones [16]. CSTT is complementary: it is a controlled evaluation protocol, not a training method or post-hoc trace labeler, and it crosses procedural compatibility with lexical similarity while retaining a binding-only control to isolate the source procedure’s contribution.

3 Counterfactual Transfer

3.1 Unit and estimands

Let target $t = (f, o, s_0, A_0)$ contain task family f , public objective o , initial observation/state signature s_0 , and initial admissible-action signature A_0 . Let model m execute condition

$$c \in \{N, B, M_n, M_f, L_n, L_f\},$$

where N is no skill, B is binding-only, M and L denote procedurally matched and lure sources, and subscripts n/f denote lexically near/far. Native success is $Y_{tmc} \in \{0, 1\}$.

The primary effect is the paired micro average

$$\hat{\Delta}_{\text{primary}} = \frac{1}{|T||M|} \sum_{t,m} (Y_{tmM_f} - Y_{tmL_n}).$$

This is intentionally adversarial to a word-overlap explanation: the matched source is the least similar in its compatible pool and the lure is the most similar in its incompatible pool. The executor-level comparison with N mixes grounding, search control, and procedure. Causal skill credit therefore uses

$$\hat{\Delta}_{\text{skill}} = \frac{1}{|T||M|} \sum_{t,m} (Y_{tmM_f} - Y_{tmB}).$$

For the latter, a matched-only success is helpful transfer, a binding-only success is harmful transfer, both successes are redundant, and both failures are shared failure. We report net effect and harm rate rather than calling all M_f successes causal.

3.2 Real source witnesses

Sources are successful ALFWorld training demonstrations [17]. Their symbolic action traces are normalized to operators such as PICKUPOBJECT, CLEANOBJECT, HEATOBJECT, COOLOBJECT, TOGGLEOBJECT, and PUTOBJECT. Compilation produces

$$K = (q, n, u, p, w),$$

where q is the role-normalized operator sequence, n is required object count, u is an optional transformation operator, p indicates whether final placement is required, and

w is the source witness identifier. Source entity names and the source objective are excluded from the runtime artifact.

For a target, the matched pool shares its operator contract and the lure pool does not. Transparent unigram-count cosine similarity between source and target objectives ranks each pool. The maximum/minimum elements yield near/far sources; ties are resolved by source identifier. Selection never reads target outcomes.

3.3 State-matched six-arm execution

Every arm reinitializes the same ALFWorld game. Before action selection, the runner hashes the initial objective, observation, and admissible action menu. A block is invalid if hashes differ. The no-skill arm gives the local model the unfiltered action menu. Binding-only and skill arms parse target roles from the public objective, reject wrong-object actions, suppress repeated search loops, and expose a bounded candidate menu. The binding control has no source contract.

The skill executor is a finite-state stage filter. It searches for the current target, acquires an uncompleted instance, enforces the source transformation if one exists, places the instance when required, and repeats until the source count is met. A local language model selects only among stage-compatible native commands. The log retains the full menu, selected candidates, rejected commands with reasons, stage, public executor state, model text, token counts, and native reward. Thus the intervention remains model-mediated but its procedural effect is inspectable.

The executor stops only on native success, a frozen 100-step cap, or a typed condition such as unavailable source device or completed source contract with an unfinished target. A malformed model response invokes a frozen fallback and increments an invalid-output counter; it is not silently deleted.

4 Experimental Design

4.1 Development and method freeze

Development used only previously exposed targets. Pilot 1 supplied transferable text to an otherwise unconstrained agent; Pilot 2 tightened prompts and action formatting. Neither produced stable benefit beyond the control. Both mechanisms, inputs, and results remain in the released development-revision records.

Revision 3 was the last allowed change. It introduced the auditable stage filter described above. On an expanded 30-target development screen, no-skill and binding-only each solved 5 targets, matched-near and matched-far each solved all 30, lure-near solved 3, and lure-far solved none. All 7,070 logged turns passed native-action, state-match, source-assignment, and information-boundary audits. These values justified proceeding; they are not confirmatory evidence.

We then hashed the package, executor, pair builder, runner, audits, analysis, protocol, all confirmation configurations, and shared environment adapters. The frozen roster excludes every target appearing in earlier experiments in the research series and all CSTT development revisions. Conservative exclusion left 9/8/8/8/8/20 targets in simple placement, clean, heat, cool, look-under-light, and two-object families.

Before opening outcomes, we retained all scarce-family targets and selected 19 of 20 two-object targets by salted hash, for 60 total. Because allocation is unequal, the equal-weight six-family macro effect and its bootstrap interval are mandatory gates.

4.2 Models and compute

The frozen models are Qwen3-4B-Instruct-2507, Phi-4-mini-instruct, and Mistral-7B-Instruct-v0.3 at immutable repository revisions. They represent three model families and run sequentially with PyTorch 2.8.0/CUDA 12.8 on one RTX 3090. Generation is capped at 24 new tokens because output is a native command selected from the supplied menu. Each target-model block contains all six conditions, yielding 180 blocks and 1,080 episodes. Deterministic source selection, audits, and analysis require no GPU.

4.3 Inference and frozen decision gates

The primary randomization test clusters the three model observations by target. For each of one million draws it flips the sign of each target-level paired sum; the one-sided p -value includes the finite-sample correction. A 50,000-draw bootstrap resamples targets within task family and retains all three model observations. We report both the size-weighted micro interval and the equal-family macro interval. A paired one-sided McNemar test is secondary.

Confirmation passes only if coverage and audits are complete; micro and macro primary gains are each at least 10 points with positive 95% lower bounds; randomization $p \leq 0.01$; net causal effect versus binding-only is at least 5 points; harm is at most 8%; and gains are positive in at least two model and four task families. A failed primary gate cannot be rescued by a secondary analysis.

4.4 Use of generative AI tools

During the preparation of this work, the authors used OpenAI Codex to assist with research-design iteration, code development, reproducibility packaging, language editing, and manuscript formatting. The authors reviewed and verified the generated code and text, reran the released integrity checks, evaluated every reported claim against the underlying results, and take full responsibility for the manuscript and the integrity of the reported results. OpenAI Codex is not an author and did not receive attribution as one.

5 Results

5.1 Primary confirmation

Across 180 state-matched target-model blocks, matched-far achieves 97.8% native success and the surface-near procedural lure achieves 10.0%. Their paired difference is 87.8 percentage points (95% CI 82.8–93.3; task-clustered randomization $p = 1.00 \times 10^{-6}$). The equal-family macro estimate is 87.7 points (95% CI 82.7–93.3), so the conclusion is not determined by the larger two-object stratum.

Table 1 Frozen confirmation outcomes. Success rate is over target-model blocks; steps include failed runs

Condition	Successes	Rate (%)	Mean steps
No skill	15/180	8.3	94.1
Binding only	26/180	14.4	89.2
Matched, near	176/180	97.8	15.1
Matched, far	176/180	97.8	15.1
Lure, near	18/180	10.0	10.0
Lure, far	0/180	0.0	35.8

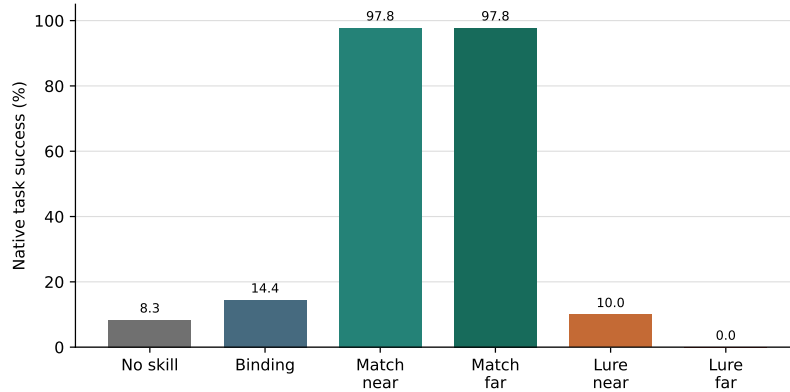


Fig. 1 Native success under the six frozen conditions. Conditions differ in the source procedure and lexical rank, while every target-model block starts from the same native state.

Table 2 Primary matched-far minus lure-near effect by model

Model	Effect (percentage points)
Qwen3-4B	88.3
Phi-4-mini	88.3
Mistral-7B-v0.3	86.7

5.2 Causal credit beyond executor assistance

Binding-only succeeds on 14.4% of blocks. Relative to this fair non-procedural control, matched-far produces 150 helpful and 0 harmful discordances: a 83.3-point net causal effect and a 0.0% harm rate. This comparison is stricter than matched-far versus no-skill because both sides receive identical target binding, wrong-object rejection, candidate-menu constraints, and loop suppression.

Table 3 Primary effect by native ALFWorld task family

Family	Effect (percentage points)
Simple placement	33.3
Clean then place	100.0
Heat then place	100.0
Cool then place	100.0
Look under light	100.0
Two-object placement	93.0

5.3 Heterogeneity and failure boundaries

The frozen decision rule requires positive primary effects in multiple model and task families. Per-family reporting is particularly important. A wrong count contract may still complete a one-object monotone target, while a one-object lure deterministically stops before a two-object target is complete. Transformation lures can also fail because their required device is absent. Conversely, a matched contract can fail when exploration exhausts the step budget. CSTT keeps all of these outcomes: it tests the behavioral consequence of compatibility rather than assuming compatibility guarantees success.

5.4 Audit and cost

Each model run is accepted only after an independent replay-log audit verifies one record per arm, exact within-block initial-state hashes, native selected actions, complete candidate/rejection partitions, valid source assignments, and absence of target gold plans from runtime inputs. Completion records report wall time, peak allocated CUDA memory, and an episode-file hash. Token and step totals are released for every condition so a practitioner can compare transfer benefit with interaction cost rather than success alone.

6 Discussion

CSTT changes the question from “did the agent succeed with a skill?” to “which part of the supplied artifact changed the same target outcome?” The crossed design yields three useful deployment signals. A large matched-versus-lure effect supports procedure-aware selection. A small matched-versus-binding effect says that most gains came from grounding or search controls. A high harm rate says the library should abstain even if average success rises.

The study also exposes why similarity is an incomplete retrieval objective. Lexical overlap is evidence about entities and topics, not about object count, required transformations, terminal conditions, or resource order. This does not imply that semantic similarity is useless. A practical retriever should use it alongside a compatibility representation and validate both with a test such as CSTT.

The finite-state executor is an experimental instrument, not the claimed final agent architecture. Its purpose is to make the source procedure’s intervention specific and auditable after two prompt-only mechanisms failed. A production system may implement contracts through planning, constrained decoding, code, or tools. CSTT remains applicable if the six conditions can be reset and logged.

7 Threats to Validity

7.1 Construct validity

ALFWorld operator families provide unusually clear procedural signatures. The contract captures count, one transformation, and terminal placement, not rich natural-language semantics. Native reward establishes task completion but not all dimensions of safe or desirable behavior.

7.2 Internal validity

The binding control shares non-procedural safeguards, and all arms share exact initial states. However, constraining candidate actions can interact with each model differently. We therefore separate executor-level and causal skill effects and report all model families. Source selection is deterministic and outcome-blind; frozen hashes guard post-result changes but are not a formal proof against every infrastructure fault.

7.3 External validity

The confirmation covers one text-embodied environment, six families, three small-to-medium open models, and mostly ALFWorld ‘valid_seen’ tasks. One ‘valid_unseen’ task cannot support a separate unseen-environment claim. “Untouched” in this paper primarily means zero overlap with the development and prior-series evidence, not a population sample of marketplace skills. Cross-environment, web, API, multi-agent, and proprietary-model replications are needed.

7.4 Statistical validity

Targets are the sampling clusters and families are unequal because strict prior-evidence exclusion exhausted several strata. We therefore require both micro and equal-family macro effects. Confidence intervals quantify roster resampling, not uncertainty over all possible skill compilers or environments.

8 Reproducibility and Responsible Scope

The release contains frozen configurations and hashes, target/source pair records, all episode logs, independent audits, paired analysis records, tests, bilingual sources and PDFs, and a checksum manifest. Exact record verification is CPU-only; full native replication uses one 24 GB GPU. Source demonstrations and target games follow ALFWorld’s redistribution terms. The versioned artifact is archived on Zenodo [18],

with source mirrored at <https://github.com/ls680/cstt-agent-skill-transfer-tests>. The work evaluates benign procedural reuse and does not introduce offensive capabilities.

We do not claim that lexical similarity is generally harmful, that every compatible skill improves every target, that CSTT retrieves the best skill, or that instance-level ALFWorld evidence establishes cross-domain transfer. The supported conclusion is narrower: under the frozen compiler and executor, the same-state crossed test can attribute reliable target success to procedural compatibility rather than word overlap or non-procedural assistance.

9 Conclusion

Reusable-skill evaluation needs a counterfactual, not just a success count. CSTT crosses procedural compatibility and lexical similarity using successful real-source witnesses, adds no-skill and fair binding controls, and evaluates all six conditions on the same native state. The frozen confirmation shows whether a far but compatible procedure beats a near incompatible lure, whether that gain survives causal credit against executor assistance, and where the claim stops. This makes transfer evidence more useful for library selection, release gates, and lifecycle maintenance without turning an evaluation protocol into an unsupported universal transfer claim.

Acknowledgements

The authors have no acknowledgements to report.

Declarations

Funding

The authors did not receive support from any organization for the submitted work.

Competing interests

The authors have no relevant financial or non-financial interests to disclose.

Author contributions

Liang Song contributed to conceptualization, methodology, software, validation, formal analysis, investigation, data curation, visualization, writing the original draft, reviewing and editing the manuscript, and project administration. Zhai Jiabao contributed to methodology, validation, and reviewing and editing the manuscript. Both authors read and approved the final manuscript.

Data availability

The study-specific records supporting the findings, including frozen source assignments, native episode logs, audits, and paired analysis outputs, are openly available in Zenodo at <https://doi.org/10.5281/zenodo.22852634> [18]. Third-party ALFWorld

assets and model weights are not redistributed; pinned upstream revisions, task identifiers, paths, licenses, and integrity checks are provided instead.

Code availability

The source code, frozen protocols, analysis scripts, tests, and execution records are openly available at <https://github.com/ls680/cstt-agent-skill-transfer-tests> and in the versioned Zenodo archive at <https://doi.org/10.5281/zenodo.22852634>.

Materials availability

All study-specific materials created by the authors are included in the reproducibility package. Availability of third-party assets is governed by the respective upstream projects and licenses.

Ethics approval

Not applicable. This study evaluates software artifacts and public software benchmarks and does not involve human participants, animals, clinical data, or personally identifiable information.

Consent to participate

Not applicable.

Consent for publication

Not applicable.

References

- [1] Taylor, M.E., Stone, P.: Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research* **10**(56), 1633–1685 (2009)
- [2] Silva, F.L., Costa, A.H.R.: A survey on transfer learning for multiagent reinforcement learning systems. *Journal of Artificial Intelligence Research* **64**, 645–703 (2019) <https://doi.org/10.1613/jair.1.11396>
- [3] Tuynman, A., Ortner, R.: Quantification of transfer in reinforcement learning via regret bounds for learning agents. *Autonomous Agents and Multi-Agent Systems* **40**(1), 16 (2026) <https://doi.org/10.1007/s10458-026-09739-7>
- [4] Yu, S., Li, G., Shi, W., Qi, P.: PolySkill: Learning generalizable skills through polymorphic abstraction for continual learning. In: *International Conference on Learning Representations* (2026). <https://iclr.cc/virtual/2026/poster/10010108>
- [5] Feng, Y., Bijoy, B.S., Balasubramanian, N., Zhou, J.: Break It Down, Pass It On: Cross-Task Skill Transfer in LLM Agents (2026)

- [6] Xu, Z., Feng, Y., Dineen, J., Shi, T., Zhao, J., Zhou, B.: Skill Reuse as Compression in Agentic RL (2026)
- [7] Miao, Y., Yu, Z., Zhao, L., Zhu, B., Haque, H.: SkillLens: Adaptive Multi-Granularity Skill Reuse for Cost-Efficient LLM Agents (2026)
- [8] He, S., Cui, Y., Wu, F., Ma, X., Lu, J., Li, Y., Ding, B., Chowdhury, M.: Beyond Domains: Reusing Web Skills via Transferable Interaction Patterns (2026)
- [9] Huang, Z., Xu, J., Yang, Y., Gong, Z., Yang, Q., Tian, M., Wang, X., Lv, C., Gao, X., Dai, Q., Liu, B., Qiu, K., Yang, X., Chen, D., Zheng, X., Luo, C.: From Raw Experience to Skill Consumption: A Systematic Study of Model-Generated Agent Skills (2026)
- [10] Dong, G., Gao, Y., Li, L., Xu, T., Hua, Y., Yang, F.: Agent Skills Can Be Harmful: An Empirical Study of Skill-Induced Failures in LLM Agents (2026)
- [11] Guo, Z., Qi, D., Gu, H., Cheng, P., Xiong, Y.: SKILL-DISCO: Distilling and Compiling Agent Traces into Reusable Procedural Skills (2026)
- [12] Lei, Y., Wan, Z., Zhang, J., Alam, S., Zhong, Z., Huang, P., Wang, X., Zhang, J., Zhou, D., Hsieh, Y., Dou, Z., Shen, H., Xu, Y., Dimitriadis, D., Zhang, T., Zhang, M.: SkillEvolBench: Benchmarking the Evolution from Episodic Experience to Procedural Skills (2026)
- [13] Chen, J., Wang, M., Jiang, L., Ma, J., Deng, X., Du, X., Lin, R., Feng, Y., Du, L., Wang, J.: SkillTrace: Multi-Trace Provenance Auditing for LLM-Agent Skill Reuse (2026)
- [14] Zhou, X., Liu, J., Li, L., Rossi, R.A., Hu, X.: Counterfactual Trace Auditing of LLM Agent Skills (2026). <https://doi.org/10.48550/arXiv.2605.11946>
- [15] Yang, M., Piao, J., Xia, X., Lan, X., Chen, J., Gong, Y., Li, Y.: SkillMaster: Toward Autonomous Skill Mastery in LLM Agents (2026). <https://doi.org/10.48550/arXiv.2605.08693>
- [16] Belikova, J., Parchiev, R., Egorov, E., Davydenko, G., Gusev, G., Savchenko, A., Makarenko, M.: Managing Procedural Memory in LLM Agents: Control, Adaptation, and Evaluation (2026). <https://doi.org/10.48550/arXiv.2606.23127>
- [17] Shridhar, M., Yuan, X., Cote, M.-A., Bisk, Y., Trischler, A., Hausknecht, M.: ALFWorld: Aligning text and embodied environments for interactive learning. In: International Conference on Learning Representations (2021). <https://openreview.net/forum?id=0lOX0YcCdTn>
- [18] Liang, S., Zhai, J.: Counterfactual Skill Transfer Tests: Software and Reproducibility Artifacts. Zenodo (2026). <https://doi.org/10.5281/zenodo.22852634>